Guía del estudiante: Aplicación web con C++ y SQLite

Oct 26, 2026 · @Leo

Introducción

Al terminar esta guía usted podrá construir, desde cero, una aplicación web completa: una página en el navegador, un servidor escrito en C++ y una base de datos SQLite que guarda la información en un archivo.

Para llegar ahí, primero va a **usar**, luego a **estudiar** y finalmente a **imitar** un proyecto que ya funciona: el **Registro de Estudiantes**. Ese proyecto permite crear, buscar, editar y eliminar estudiantes, y muestra estadísticas como el promedio y la cantidad de aprobados.

Qué va a aprender

- Cómo se comunican un navegador y un servidor (modelo cliente-servidor y protocolo HTTP).
- Qué es una API REST y cómo se diseña.
- Cómo guardar y consultar datos con SQL en una base de datos SQLite.
- Cómo organizar un programa en C++ en capas: modelo, acceso a datos y rutas.
- Cómo una página web usa JavaScript para pedir y mostrar datos.
- Buenas prácticas de seguridad: validación en el servidor, consultas con parámetros y prevención de XSS.

Cómo usar esta guía

La guía está dividida en fases. Hágalas **en orden**: cada una usa lo aprendido en la anterior.

Fase	Qué hace usted	Resultado
0. Preparar el entorno	Instala el compilador y un editor	Puede compilar programas C++
1. Correr el proyecto	Compila y ejecuta el demo	La aplicación funciona en su navegador

Fase	Qué hace usted	Resultado
2. Conceptos base	Lee la teoría mínima necesaria	Entiende el vocabulario
3. Mapa del proyecto	Recorre las carpetas y archivos	Sabe dónde está cada cosa
4. Estudio del código	Lee el código capa por capa	Entiende cómo funciona por dentro
5. Laboratorio guiado	Modifica el demo	Demuestra que lo entendió
6. Metodología	Aprende el orden para construir	Tiene una receta paso a paso
7. Proyecto final	Construye su propia aplicación	Su proyecto entregable

A lo largo de la guía encontrará casillas como esta, para que marque su avance:

- [ ] Leí la introducción y entiendo qué voy a construir.

Fase 0 – Preparar el entorno

Para compilar el proyecto solo necesita el compilador **g++** (de MSYS2) y un editor de código. Las librerías ya vienen incluidas dentro del proyecto.

Paso 1: instalar MSYS2 y g++

1. Descargue el instalador desde msys2.org e instálelo en la ruta por defecto (`C:\msys64`).
2. Al terminar se abre una terminal llamada **MSYS2 UCRT64**. Escriba este comando y presione Enter (acepte con `Y` cuando pregunte):

```
pacman -S mingw-w64-ucrt-x86_64-gcc mingw-w64-ucrt-x86_64-make
```

Paso 2: agregar g++ al PATH de Windows

El PATH es la lista de carpetas donde Windows busca programas. Si `g++` no está ahí, Windows no lo encuentra.

1. Presione la tecla Windows y busque **"Editar las variables de entorno del sistema"**.

2. Clic en **Variables de entorno…**.
3. En *Variables de usuario*, seleccione **Path** → **Editar** → **Nuevo**.
4. Escriba `C:\msys64\ucrt64\bin` y acepte todas las ventanas.
5. **Cierre y vuelva a abrir** cualquier terminal que tuviera abierta.

Paso 3: verificar

Abra una terminal nueva (PowerShell o CMD) y escriba:

```
g++ --version
```

Si aparece algo como `g++.exe (Rev5, Built by MSYS2 project) ...`, todo está bien. Si dice *"no se reconoce como un comando"*, revise el Paso 2.

Paso 4: editor recomendado

Use **Visual Studio Code** (code.visualstudio.com) con la extensión **C/C++** de Microsoft. Abra la carpeta completa del proyecto con *Archivo → Abrir carpeta*, no archivos sueltos.

Opcional pero muy útil: **DB Browser for SQLite** (sqlitebrowser.org) para ver el contenido de la base de datos.

- [ ] Instalé MSYS2 y g++.
- [ ] `g++ --version` funciona en una terminal nueva.
- [ ] Tengo VS Code (y opcionalmente DB Browser for SQLite).

Fase 1 – Correr el proyecto

Antes de leer una sola línea de código, ponga la aplicación a funcionar y úsela. Es mucho más fácil entender un programa cuando ya sabe qué hace.

Paso 1: compilar

Haga doble clic en **`compilar.bat`**, dentro de la carpeta del proyecto. Verá tres etapas:

```
[1/3] Compilando SQLite (solo la primera vez, puede tardar)...
[2/3] Compilando codigo C++...
[3/3] Enlazando...
Listo: build\servidor.exe
```

La primera vez tarda alrededor de un minuto porque compila SQLite, que tiene más de 250 000 líneas de código C. Las siguientes veces tarda pocos segundos.

> **Compilar** significa traducir el código C++ (texto que usted entiende) a un archivo `.exe` (instrucciones que entiende la computadora).

Paso 2: ejecutar

Haga doble clic en **`iniciar.bat`**. Se abre una ventana negra (la consola del servidor) y el navegador en `http://localhost:8080`.

- `localhost` significa "esta misma computadora".
- `8080` es el **puerto**: el número de "puerta" por donde el servidor escucha.

No cierre la ventana negra mientras usa la aplicación: si la cierra, el servidor se detiene. Para detenerlo a propósito, presione `Ctrl + C` dentro de ella.

Paso 3: explorar la aplicación

Haga cada una de estas acciones y, después de cada una, mire la ventana negra. Verá una línea por cada petición que recibe el servidor, por ejemplo `POST /api/estudiantes -> 201`.

- [ ] Ver la lista de 6 estudiantes de ejemplo y las estadísticas.
- [ ] Crear un estudiante nuevo con el formulario.
- [ ] Buscar escribiendo "11-2" en el buscador.
- [ ] Editar la nota de un estudiante.
- [ ] Eliminar un estudiante.
- [ ] Intentar guardar una nota de 150 y observar el mensaje de error.
- [ ] Cerrar el servidor, volver a abrirlo y comprobar que los datos **siguen ahí**.

El último punto demuestra que los datos se guardan en un archivo (`data/escuela.db`) y no en la memoria del programa.

¿Cuándo hay que volver a compilar?

Si cambia…	¿Recompilar?	Qué hacer
Nada, solo quiere usar la app	No	`iniciar.bat`

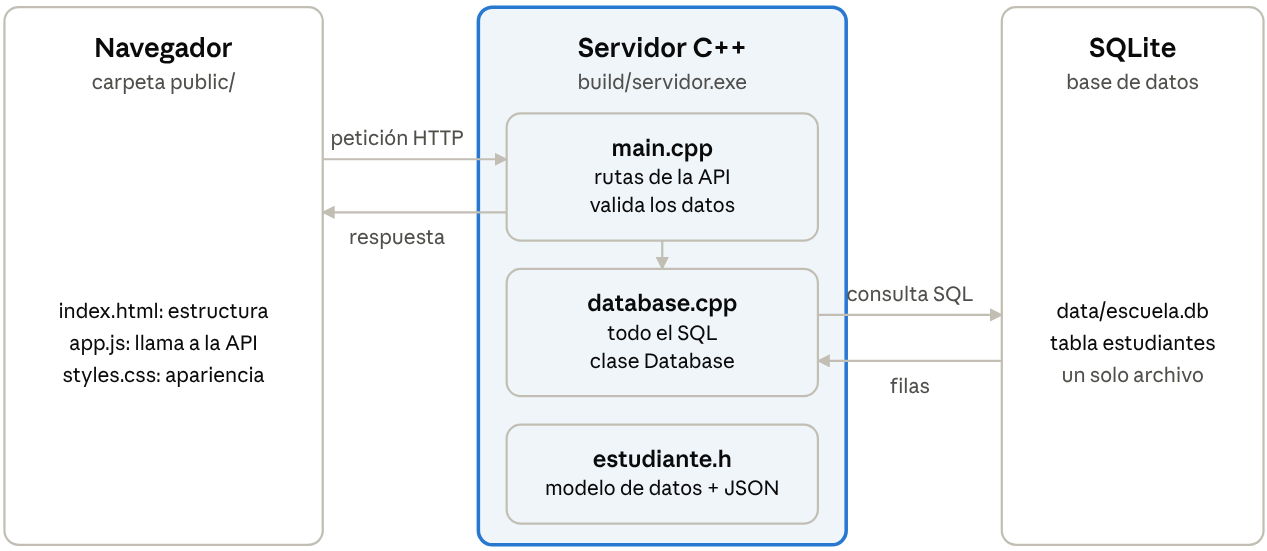
Si cambia…	¿Recompilar?	Qué hacer
Archivos de `public/` (HTML, CSS, JS)	No	Recargue el navegador con F5
Archivos de `src/` (`.cpp` o `.h`)	Sí	Detenga el servidor (`Ctrl + C`), ejecute `compilar.bat` y luego `iniciar.bat`
Quiere empezar con la base de datos limpia	No	Detenga el servidor y borre `data/escuela.db`

Regla para recordar: C++ es un lenguaje *compilado* (hay que traducirlo cada vez que cambia). HTML, CSS y JavaScript los interpreta el navegador directamente.

Fase 2 – Conceptos base

La aplicación tiene tres partes que se pasan información en cadena: el navegador le pide datos al servidor C++, y el servidor se los pide a la base de datos SQLite. El navegador **nunca** toca la base de datos directamente.

El navegador nunca toca la base de datos: todo pasa por el servidor



Arquitectura del demo · 3 partes y los archivos de cada una

El servidor (en color) es el único intermediario: recibe peticiones HTTP del navegador, las traduce a consultas SQL y devuelve las filas como JSON.

Cliente y servidor

- El **cliente** es el navegador. Muestra la página y envía peticiones.
- El **servidor** es nuestro programa `servidor.exe`. Recibe peticiones, decide qué hacer y responde.
- El servidor es quien manda: decide qué datos son válidos y qué está permitido. Por eso **toda validación importante se hace en el servidor**, aunque el formulario también valide.

HTTP: el idioma entre cliente y servidor

Cada petición HTTP tiene un **método** (qué se quiere hacer) y una **ruta** (sobre qué). La respuesta trae un **código de estado** (cómo salió) y un **cuerpo** (los datos).

Método	Significado	Equivalente en SQL
`GET`	Leer	`SELECT`
`POST`	Crear	`INSERT`
`PUT`	Actualizar	`UPDATE`
`DELETE`	Eliminar	`DELETE`

Código	Significado
`200`	Todo bien
`201`	Se creó algo nuevo
`400`	El cliente envió datos inválidos
`404`	No existe lo que se pidió
`500`	Error interno del servidor

API REST

Una **API** es la lista de peticiones que un servidor sabe atender. Es **REST** cuando cada tipo de dato tiene una ruta (`/api/estudiantes`) y se usan los métodos HTTP para indicar la acción. Por ejemplo, `DELETE /api/estudiantes/3` significa "elimine al estudiante con id 3".

JSON: el formato de los datos

JSON es texto con una estructura fácil de leer para personas y programas. Así viaja un estudiante entre el navegador y el servidor:

```
{ "id": 1, "nombre": "Ana", "apellido": "Rodríguez Mora", "seccion": "11-1",
"nota": 95 }
```

- `{ }` es un **objeto** (como un `struct` de C++).
- `[ ]` es un **arreglo** (como un `vector`).

Base de datos y SQL

Una **base de datos relacional** guarda información en **tablas**: cada fila es un registro (un estudiante) y cada columna un atributo (nombre, nota…). **SQL** es el lenguaje para consultarla. **SQLite** es un motor de base de datos que guarda todo en un solo archivo y no necesita instalar un servidor aparte.

```
SELECT nombre, nota FROM estudiantes WHERE nota >= 70 ORDER BY nota DESC;
```

El recorrido completo de una petición

Esto es lo que pasa cuando usted pulsa **Guardar** para crear un estudiante:

1. `app.js` lee el formulario y arma un objeto JSON.
2. `app.js` envía `POST /api/estudiantes` con ese JSON usando `fetch()`.
3. `main.cpp` recibe la petición y **valida** los datos. Si algo está mal, responde `400`.
4. `main.cpp` llama a `db.crear(estudiante)`.
5. `database.cpp` ejecuta un `INSERT` en SQLite.
6. SQLite guarda la fila en `data/escuela.db` y devuelve el id nuevo.
7. `main.cpp` responde `201` con el estudiante creado en JSON.
8. `app.js` muestra un mensaje y vuelve a cargar la tabla.

Preguntas de comprensión

1. ¿Por qué el navegador no se conecta directamente a la base de datos?
2. ¿Qué método HTTP y qué ruta usaría para obtener al estudiante con id 4?
3. Si el servidor responde `404`, ¿de quién es el "error": del cliente o del servidor?
4. Escriba en JSON un estudiante inventado.

Fase 3 – Mapa del proyecto

El proyecto separa el código en dos mundos: `src/` es el **backend** (C++, corre en el servidor) y `public/` es el **frontend** (lo que corre en el navegador). Todo lo demás es apoyo.

```
demo1/
├── src/                    ← BACKEND (C++)
│   ├── estudiante.h        Modelo de datos + conversión a JSON
│   ├── database.h          Declaración de la clase Database
│   ├── database.cpp        Todo el SQL está aquí
│   └── main.cpp            Servidor HTTP: rutas de la API y validación
│
├── public/                 ← FRONTEND (navegador)
│   ├── index.html          Estructura de la página
│   ├── styles.css          Apariencia
│   └── app.js              Lógica: llamadas a la API y dibujo de la tabla
│
├── third_party/            ← Librerías externas (NO se modifican)
│   ├── httplib/httplib.h   Servidor HTTP
│   ├── json/json.hpp       Manejo de JSON
│   └── sqlite/sqlite3.c .h  Base de datos SQLite
│
├── data/escuela.db         Base de datos (se crea sola)
├── build/                  Archivos compilados (se crea al compilar)
├── compilar.bat            Compila en Windows
├── iniciar.bat             Arranca el servidor y abre el navegador
├── Makefile                Compilación alternativa con make
└── README.md               Documentación técnica resumida
```

Las tres librerías externas

Nadie escribe todo desde cero: usamos tres librerías probadas por miles de programadores. Las tres son de **un solo archivo**, así que basta con incluirlas.

Librería	Para qué la usamos	Se usa en
cpp-httplib 0.18.3	Recibir peticiones HTTP y responderlas	`main.cpp`
nlohmann/json 3.11.3	Convertir entre JSON y objetos C++	`estudiante.h`, `main.cpp`
SQLite 3.49.1	Guardar y consultar los datos	`database.cpp`

Orden recomendado de lectura

Lea los archivos en este orden, que va **de adentro hacia afuera**: primero los datos, después quién los guarda, después quién los expone y al final quién los muestra.

1. `src/estudiante.h` – ¿Qué datos maneja la aplicación?
2. `src/database.h` – ¿Qué operaciones se pueden hacer con esos datos?
3. `src/database.cpp` – ¿Cómo se hacen esas operaciones con SQL?
4. `src/main.cpp` – ¿Cómo se ofrecen esas operaciones por HTTP?
5. `public/index.html` – ¿Qué elementos tiene la página?
6. `public/app.js` – ¿Cómo la página llama a la API y muestra los resultados?
7. `public/styles.css` – ¿Cómo se ve? (el menos importante para entender la lógica)
8. `compilar.bat` – ¿Cómo se convierte todo en un `.exe`?

- [ ] Abrí la carpeta del proyecto en VS Code y ubiqué cada archivo de la lista.

Fase 4 – Estudio del código, capa por capa

Ahora sí, al código. Tenga cada archivo abierto en VS Code mientras lee esta sección: aquí están las partes clave explicadas, pero el archivo completo tiene más comentarios. Al final de cada capa hay preguntas; respóndalas en su cuaderno antes de seguir.

4.1 El modelo: `estudiante.h`

Un **modelo** es una estructura que representa **una fila** de una tabla. Cada campo del `struct` corresponde a una columna.

```
struct Estudiante {
    int         id = 0;      // Clave primaria (la asigna SQLite)
    std::string nombre;
    std::string apellido;
    std::string seccion;
    double      nota = 0.0;
    std::string creado_en;   // Fecha de creación (la pone SQLite)
};
```

El mismo archivo define cómo convertir un `Estudiante` a JSON. La librería nlohmann/json busca automáticamente una función llamada `to_json`:

```
inline void to_json(json& j, const Estudiante& e) {
    j = json{ {"id", e.id}, {"nombre", e.nombre}, /* ... */ };
}
```

Gracias a eso, en `main.cpp` basta con escribir `responderJson(res, estudiante)` y la conversión ocurre sola. También funciona con un `std::vector<Estudiante>`: se convierte en un arreglo JSON.

Preguntas 4.1

1. ¿Qué tipo de C++ se usa para la nota y por qué no `int`?
2. ¿Qué dos campos del `struct` no los escribe el usuario?
3. ¿Qué representa el `struct Estadisticas` del mismo archivo?

4.2 La interfaz de datos: `database.h`

Este archivo solo **declara** qué sabe hacer la clase `Database`; no dice cómo. Es el "menú" de operaciones disponibles:

```
class Database {
public:
    explicit Database(const std::string& ruta_archivo);
    ~Database();
    void inicializar();

    std::vector<Estudiante>   listar(const std::string& filtro = "");
    std::optional<Estudiante> obtener(int id);
    Estudiante                crear(const Estudiante& e);
    bool                      actualizar(int id, const Estudiante& e);
    bool                      eliminar(int id);
    Estadisticas              estadisticas();
private:
    sqlite3*   db_ = nullptr;
    std::mutex mutex_;
};
```

Ideas importantes:

- **CRUD**: Create, Read, Update, Delete. Casi cualquier aplicación de datos tiene estas cuatro operaciones.
- `std::optional<Estudiante>` significa "puede que haya un estudiante, o puede que no". Es la forma moderna de decir "no encontrado" sin usar punteros ni valores mágicos.
- `actualizar` y `eliminar` devuelven `bool`: `false` si el id no existía.
- **Separación de capas**: solo esta clase conoce SQLite. Si mañana se cambia SQLite por MySQL, solo se modifica `database.cpp`.

4.3 El acceso a datos: `database.cpp`

Aquí está todo el SQL. Primero, cómo funciona SQLite desde C/C++. Toda consulta sigue cinco pasos:

Paso	Función de SQLite	Qué hace
1	`sqlite3_prepare_v2`	"Compila" el texto SQL
2	`sqlite3_bind_*`	Rellena los parámetros `?` con valores
3	`sqlite3_step`	Ejecuta; devuelve una fila cada vez
4	`sqlite3_column_*`	Lee las columnas de la fila actual

Paso	Función de SQLite	Qué hace
5	`sqlite3_finalize`	Libera la memoria de la consulta

La clase `Sentencia` (RAII). Olvidar el paso 5 provoca fugas de memoria. Para que eso nunca pase, el proyecto envuelve los pasos en una clase: el constructor hace el paso 1 y el destructor hace el paso 5 automáticamente. Esta técnica se llama **RAII** y es una de las ideas más importantes de C++.

```
class Sentencia {
public:
    Sentencia(sqlite3* db, const std::string& sql) { /* sqlite3_prepare_v2 */
}
    ~Sentencia() { sqlite3_finalize(stmt_); }   // siempre se libera
    void bind(int i, const std::string& v);     // paso 2
    bool siguiente();                           // paso 3
    std::string texto(int col);                 // paso 4
    // ...
};
```

Crear la tabla. `inicializar()` crea la tabla si no existe e inserta 6 estudiantes de ejemplo solo si está vacía:

```
CREATE TABLE IF NOT EXISTS estudiantes (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre     TEXT    NOT NULL,
    apellido   TEXT    NOT NULL,
    seccion    TEXT    NOT NULL,
    nota       REAL    NOT NULL CHECK (nota BETWEEN 0 AND 100),
    creado_en  TEXT    NOT NULL DEFAULT (datetime('now', 'localtime'))
);
```

Un método completo, línea por línea. Así se elimina un estudiante:

```
bool Database::eliminar(int id) {
    std::lock_guard<std::mutex> lock(mutex_);              // (a)
    Sentencia s(db_, "DELETE FROM estudiantes WHERE id = ?"); // (b)
    s.bind(1, id);                                         // (c)
    s.siguiente();                                         // (d)
    return sqlite3_changes(db_) > 0;                       // (e)
}
```

- **(a)** Bloquea el *mutex*. El servidor atiende varias peticiones a la vez (en hilos distintos); el mutex garantiza que solo una use la base de datos en cada momento. Se desbloquea solo al salir de la función.
- **(b)** Prepara el SQL. El `?` es un **parámetro**: un espacio que se rellena después.
- **(c)** Rellena el primer `?` con el id. Los parámetros se cuentan desde 1.
- **(d)** Ejecuta la sentencia.
- **(e)** `sqlite3_changes` dice cuántas filas se borraron. Si fue 0, el id no existía.

Seguridad: por qué se usa `?` y no se concatena texto. Imagine que la búsqueda se hiciera así:

```
// ❌ NUNCA haga esto
std::string sql = "SELECT * FROM estudiantes WHERE nombre = '" + filtro +
"'";
```

Si alguien escribe en el buscador `' OR 1=1 --`, el SQL resultante devuelve **todos** los registros, y con otras variantes podría borrar tablas. Este ataque se llama **inyección SQL**. Con `?` y `bind`, el valor nunca se mezcla con el código SQL: SQLite lo trata siempre como dato. Pruébelo: escriba `' OR 1=1 --` en el buscador del demo y verá que no encuentra nada.

Estadísticas con funciones de agregación. Una sola consulta calcula todo:

```
SELECT COUNT(*), AVG(nota), MAX(nota), MIN(nota),
       SUM(CASE WHEN nota >= 70 THEN 1 ELSE 0 END)   -- aprobados
FROM estudiantes;
```

Preguntas 4.2 y 4.3

1. ¿Qué pasaría si dos peticiones intentaran insertar al mismo tiempo sin el mutex?
2. ¿Qué hace `CHECK (nota BETWEEN 0 AND 100)`? ¿Qué pasa si de todas formas se intenta guardar 150?
3. En `listar()`, ¿por qué el filtro se rodea con `%`?
4. Explique con sus palabras qué es RAII y dónde se usa en este archivo.
5. Abra `data/escuela.db` con DB Browser for SQLite y ejecute la consulta de estadísticas a mano.

4.4 El servidor y la API: `main.cpp`

`main()` hace cuatro cosas en orden: abre la base de datos, crea el servidor, registra las rutas y se queda escuchando.

```
Database db("data/escuela.db");   // 1. abrir la base de datos
db.inicializar();

httplib::Server svr;               // 2. crear el servidor
svr.set_mount_point("/", "public"); //    servir HTML, CSS y JS

// 3. registrar rutas (ver abajo)

svr.listen("127.0.0.1", 8080);     // 4. escuchar (se queda aquí)
```

Una ruta = método + camino + función. Cada ruta asocia una petición con una función *lambda* (una función sin nombre escrita en el lugar) que recibe la petición `req` y llena la respuesta `res`:

```
svr.Get("/api/estudiantes", [&](const httplib::Request& req,
httplib::Response& res) {
    std::string filtro = req.has_param("q") ? req.get_param_value("q") : "";
    responderJson(res, db.listar(filtro));
});
```

- `[&]` permite a la lambda usar la variable `db` declarada afuera.
- `req.get_param_value("q")` lee lo que viene en la URL después de `?q=`.

Rutas con parámetros. Para leer el id de `/api/estudiantes/5` se usa una expresión regular: `(\d+)` captura uno o más dígitos, y se lee con `req.matches[1]`.

```
svr.Delete(R"(/api/estudiantes/(\d+))", [&](const httplib::Request& req,
httplib::Response& res) {
    int id = std::stoi(req.matches[1]);
    if (!db.eliminar(id)) return responderError(res, 404, "Estudiante no
encontrado");
    responderJson(res, json{{"ok", true}});
});
```

Validación. Antes de guardar, `validarYConvertir()` revisa el JSON recibido: que sea JSON válido, que tenga todos los campos con el tipo correcto, que no estén vacíos y que la nota esté entre 0 y 100. Si algo falla, devuelve un mensaje y la ruta responde `400`. Nunca

confíe en lo que envía el navegador: cualquiera puede enviar peticiones sin usar su formulario.

La API completa:

Método	Ruta	Llama a	Respuesta
`GET`	`/api/estudiantes?q=...`	`db.listar(q)`	`200` + arreglo
`GET`	`/api/estudiantes/{id}`	`db.obtener(id)`	`200` o `404`
`POST`	`/api/estudiantes`	`db.crear(e)`	`201`, `400`
`PUT`	`/api/estudiantes/{id}`	`db.actualizar(id, e)`	`200`, `400` o `404`
`DELETE`	`/api/estudiantes/{id}`	`db.eliminar(id)`	`200` o `404`
`GET`	`/api/estadisticas`	`db.estadisticas()`	`200` + objeto

Manejo de errores. `set_exception_handler` atrapa cualquier excepción (por ejemplo, un error de SQLite) y responde `500` en vez de cerrar el programa. `set_logger` imprime en la consola cada petición recibida: eso es lo que usted vio en la ventana negra.

Preguntas 4.4

1. ¿Qué ruta y qué método de `Database` se ejecutan cuando usted pulsa **Eliminar**?
2. ¿Por qué `POST` responde `201` y no `200`?
3. ¿Qué pasa si alguien envía `{"nombre": "Ana"}` sin los demás campos?
4. Con el servidor encendido, abra en el navegador `http://localhost:8080/api/estudiantes`. ¿Qué ve? ¿Y con `/api/estudiantes/999`?

4.5 El frontend: `index.html` y `app.js`

`index.html` tiene la estructura: las tarjetas de estadísticas, el formulario y una tabla con un `<tbody id="tabla">` **vacío**. Ese cuerpo lo llena JavaScript con los datos que llegan del servidor.

La función `api()` centraliza todas las llamadas al servidor:

```
async function api(ruta, opciones = {}) {
  const resp = await fetch('/api' + ruta, {
    headers: { 'Content-Type': 'application/json' },
    ...opciones,
  });
  const datos = await resp.json();
  if (!resp.ok) throw new Error(datos.error || `Error ${resp.status}`);
  return datos;
}
```

- `fetch()` envía la petición HTTP.
- `async` / `await`: la petición tarda un poco; `await` espera la respuesta sin congelar la página.
- Si el servidor responde con error (`400`, `404`…), se lanza una excepción con el mensaje del servidor, que luego se muestra al usuario.

Usarla es una línea:

```
await api('/estudiantes', { method: 'POST', body: JSON.stringify(datos) });
// crear
await api('/estudiantes/' + id, { method: 'DELETE' });
// eliminar
```

Dibujar la tabla. `cargarEstudiantes()` pide la lista y crea una fila `<tr>` por estudiante. Usa `textContent` y no `innerHTML` para los datos: si alguien guardara como nombre `<script>...</script>`, con `innerHTML` ese código se ejecutaría (ataque **XSS**); con `textContent` se muestra como texto inofensivo.

Crear o editar con el mismo formulario. El formulario tiene un campo oculto `id`. Si está vacío, *Guardar* hace `POST` (crear); si tiene valor, hace `PUT` (editar). El botón *Editar* de cada fila solo copia los datos al formulario y llena ese id.

Búsqueda con *debounce*. Al escribir en el buscador no se hace una petición por cada letra: se espera 250 ms después de la última tecla.

Preguntas 4.5

1. Abra las herramientas de desarrollador del navegador (F12) → pestaña **Red / Network**. Cree un estudiante y busque la petición `POST`. ¿Qué JSON se envió y qué se recibió?
2. ¿Qué función de `app.js` se ejecuta al cargar la página?
3. ¿Por qué después de guardar se llama a `refrescar()`?

4.6 La compilación: `compilar.bat`

Compilar tiene dos etapas: cada archivo fuente se convierte en un archivo objeto (`.o`), y después el **enlazador** junta todos los `.o` en un solo `.exe`.

```
gcc -O2 -c third_party\sqlite\sqlite3.c -o build\sqlite3.o       :: solo la
1.ª vez
g++ %FLAGS% -c src\main.cpp     -o build\main.o
g++ %FLAGS% -c src\database.cpp -o build\database.o
g++ build\main.o build\database.o build\sqlite3.o -o build\servidor.exe -
lws2_32 -static
```

Opción	Significado
`-std=c++17`	Usar el estándar C++17 (necesario para `std::optional`)
`-O2`	Optimizar el programa
`-Wall`	Mostrar todas las advertencias
`-I carpeta`	Buscar archivos `#include` en esa carpeta
`-c`	Solo compilar a `.o`, sin enlazar
`-lws2_32`	Enlazar la librería de red de Windows
`-static`	Meter todo dentro del `.exe` para que funcione en otras computadoras

SQLite se compila con `gcc` (es código C) y una sola vez, porque tarda. Si `build\sqlite3.o` ya existe, se reutiliza.

- [ ] Leí los 8 archivos en el orden recomendado.
- [ ] Respondí todas las preguntas de la Fase 4.

Fase 5 – Laboratorio guiado

Usted entiende el proyecto cuando puede cambiarlo sin romperlo. Los ejercicios van de menor a mayor dificultad; el primero está resuelto paso a paso porque enseña el patrón que se repite en todos: **un cambio atraviesa las capas en orden, de la base de datos hacia la pantalla**.

Antes de empezar, haga una copia de la carpeta del proyecto (`demo1_original`) por si necesita volver atrás.

Ejercicio 1 (resuelto): agregar el campo `correo`

#	Capa	Archivo	Cambio
1	Base de datos	`database.cpp`	Agregar `correo TEXT NOT NULL DEFAULT ''` al `CREATE TABLE` y `correo` a la constante `COLUMNAS`
2	Modelo	`estudiante.h`	Agregar `std::string correo;` al `struct` y `{"correo", e.correo}` en `to_json`
3	Acceso a datos	`database.cpp`	Leer la columna nueva en `leerEstudiante()` (¿qué número de columna le toca?) y agregarla al `INSERT` y al `UPDATE` con su `?` y su `bind`
4	API	`main.cpp`	En `validarYConvertir()` , leer `correo` del JSON y validar que contenga `@`
5	Interfaz	`index.html`	Agregar un `<input id="correo" type="email">` al formulario y una columna `<th>Correo</th>` a la tabla
6	Lógica del cliente	`app.js`	Enviar `correo` en `datos` , mostrarlo en la tabla y cargarlo en `editar()`

Después:

1. **Borre `data/escuela.db`.** `CREATE TABLE IF NOT EXISTS` no modifica una tabla que ya existe, así que la tabla vieja (sin `correo`) seguiría ahí.
2. Recompile con `compilar.bat` y ejecute con `iniciar.bat` .
3. Pruebe crear, editar y buscar un estudiante con correo. Pruebe también un correo sin `@` .

> **Consejo:** si al compilar aparece un error, lea la **primera** línea que diga `error:` . Indica el archivo y la línea exacta. Los errores siguientes suelen ser consecuencia del primero.

Ejercicios para resolver

#	Ejercicio	Capas que toca	Pista
2	Cambiar la nota mínima de aprobación de 70 a 65	Datos, cliente	Busque `70` en todo el proyecto (Ctrl + Shift + F en VS Code)
3	Agregar la tarjeta "Nota máxima" a las estadísticas	Interfaz, cliente	El servidor ya envía `nota_max`
4	Nueva ruta `GET /api/estadisticas/secciones` con el promedio por sección	Datos, API	`SELECT seccion, AVG(nota), COUNT(*) FROM estudiantes GROUP BY seccion`
5	Mostrar la tabla de promedios por sección en la página	Interfaz, cliente	Use la ruta del ejercicio 4
6	Ordenar la tabla al hacer clic en un encabezado	Datos, API, cliente	Parámetro `?orden=nota`. No se puede usar `?` para nombres de columna: use una **lista blanca** de columnas permitidas
7	Paginación de 5 en 5	Datos, API, cliente	`LIMIT ? OFFSET ?` y botones *Anterior* / *Siguiente*
8	Segunda tabla `secciones` (id, nombre, profesor) relacionada con `estudiantes`	Todas	`FOREIGN KEY` y `JOIN`

- [ ] Ejercicio 1 funcionando.
- [ ] Al menos 4 de los ejercicios 2 a 8 resueltos.

Fase 6 – Metodología para construir un proyecto desde cero

El demo no se escribió de una sola vez: se construyó en pasos pequeños, **diseñando primero y programando después**, y probando cada pieza antes de pasar a la siguiente. Siga estos 10 pasos en orden. En los ejemplos se usa un proyecto imaginario de **biblioteca** con una tabla `libros`; cámbielo por su tema.

> **Regla de oro:** compile y pruebe después de cada paso. Si escribe 200 líneas y luego compila, tendrá 30 errores mezclados. Si compila cada 20 líneas, tendrá uno y sabrá dónde está.

Paso 1: definir el problema (en papel)

Responda por escrito, antes de abrir VS Code:

- ¿Qué problema resuelve la aplicación y para quién?
- ¿Cuál es la **entidad** principal? (estudiante, libro, producto, cita…)
- ¿Qué datos tiene cada una? ¿Cuáles son obligatorios?
- ¿Qué operaciones necesita el usuario? (como mínimo: CRUD + búsqueda)
- ¿Qué reglas deben cumplir los datos? (rangos, campos no vacíos, formatos)

Paso 2: diseñar la tabla

Llene esta plantilla para su entidad:

Columna	Tipo SQL	Restricciones	Ejemplo
`id`	`INTEGER`	`PRIMARY KEY AUTOINCREMENT`	1
`titulo`	`TEXT`	`NOT NULL`	Cien años de soledad
`autor`	`TEXT`	`NOT NULL`	Gabriel García Márquez
`anio`	`INTEGER`	`CHECK (anio BETWEEN 1000 AND 2100)`	1967
`disponible`	`INTEGER`	`NOT NULL DEFAULT 1` (SQLite usa 0/1 para sí/no)	1

Luego **pruebe el SQL antes de programar**: en DB Browser for SQLite cree una base de datos nueva, ejecute su `CREATE TABLE`, inserte 3 filas y escriba los `SELECT`, `UPDATE` y `DELETE` que va a necesitar. Si el SQL funciona ahí, funcionará en C++.

Paso 3: diseñar la API

Llene esta plantilla. Siga el mismo patrón REST del demo:

Método	Ruta	Cuerpo que recibe	Responde	Método de `Database`
`GET`	`/api/libros?q=`	—	`200` + arreglo	`listar(q)`
`GET`	`/api/libros/{id}`	—	`200` / `404`	`obtener(id)`
`POST`	`/api/libros`	libro sin `id`	`201` / `400`	`crear(l)`
`PUT`	`/api/libros/{id}`	libro sin `id`	`200` / `400` / `404`	`actualizar(id, l)`
`DELETE`	`/api/libros/{id}`	—	`200` / `404`	`eliminar(id)`

Paso 4: crear el esqueleto del proyecto

1. Cree una carpeta nueva para su proyecto.
2. Copie del demo, **sin cambios**: la carpeta `third_party/` completa, `compilar.bat` e `iniciar.bat`.
3. Cree las carpetas `src/` y `public/`.
4. Cree estos archivos **vacíos**: `src/libro.h`, `src/database.h`, `src/database.cpp`, `public/index.html`, `public/app.js`, `public/styles.css`. (Un `.cpp` vacío compila sin problema; así `compilar.bat` funciona desde el primer momento.)

Paso 5: servidor mínimo ("hola mundo")

Escriba este `src/main.cpp` mínimo y escriba `<h1>Funciona</h1>` en `public/index.html`:

```
#include <iostream>
#include "httplib.h"

int main() {
    httplib::Server svr;
    svr.set_mount_point("/", "public");

    svr.Get("/api/ping", [](const httplib::Request&, httplib::Response& res)
{
        res.set_content("{\"mensaje\": \"pong\"}", "application/json");
    });

    std::cout << "Servidor en http://localhost:8080\n";
    svr.listen("127.0.0.1", 8080);
}
```

Compile, ejecute y verifique que `http://localhost:8080` muestra "Funciona" y que `http://localhost:8080/api/ping` muestra el JSON. **No siga hasta que esto funcione.**

Paso 6: el modelo

En `src/libro.h` escriba el `struct` con un campo por columna del Paso 2 y su función `to_json`. Use `estudiante.h` del demo como guía. Compile.

Paso 7: la base de datos, una operación a la vez

Esta es la parte más larga. Use `database.h` y `database.cpp` del demo como modelo y copie la clase `Sentencia` tal cual (sirve para cualquier proyecto). Avance en este orden, **conectando cada método a su ruta y probándolo antes de escribir el siguiente**:

1. Constructor, destructor e `inicializar()` (con `CREATE TABLE` y 3 filas de ejemplo; antes, en main(), cree la carpeta data con std::filesystem::create_directories("data") como hace el demo, porque SQLite no crea carpetas). Compile, ejecute y verifique con DB Browser que el archivo `.db` tiene la tabla.
2. `listar()` + ruta `GET /api/libros`. Pruébela en el navegador.
3. `obtener()` + ruta `GET /api/libros/{id}`. Pruebe un id que existe y uno que no (`404`).
4. `crear()` + ruta `POST`.
5. `actualizar()` + ruta `PUT`.
6. `eliminar()` + ruta `DELETE`.

7. Búsqueda con `?q=` dentro de `listar()`.

¿Cómo probar `POST`, `PUT` y `DELETE` sin tener todavía la página? Abra `http://localhost:8080`, presione F12 → pestaña **Consola** y escriba:

```
fetch('/api/libros', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ titulo: 'Prueba', autor: 'Yo', anio: 2026 })
}).then(r => r.json()).then(console.log)
```

Cambie `method`, la ruta y el cuerpo para probar las demás operaciones.

Paso 8: validación y errores

Agregue al servidor su versión de `validarYConvertir()` con las reglas del Paso 1, y las funciones `responderJson()` / `responderError()` y `set_exception_handler` del demo. Pruebe desde la consola enviar datos malos: campos vacíos, tipos incorrectos, valores fuera de rango, texto que no es JSON. Todos deben responder `400` con un mensaje claro.

Paso 9: el frontend, de a poco

1. `index.html`: el formulario y una tabla con `<tbody>` vacío. Sin JavaScript todavía.
2. `app.js`: copie la función `api()` del demo. Luego programe, probando cada una antes de seguir: **listar** → **crear** → **eliminar** → **editar** → **buscar**.
3. `styles.css`: el aspecto visual va **al final**, cuando todo funciona.

Paso 10: documentar y entregar

1. Revise que cada archivo tenga un comentario inicial que explique para qué sirve, y que cada función no obvia tenga un comentario.
2. Escriba un `README.md` con: descripción, cómo compilar y ejecutar, estructura de carpetas, la tabla de la base de datos y la tabla de la API (las de los Pasos 2 y 3).
3. Haga la prueba final: borre `build/` y `data/`, compile desde cero, ejecute y pruebe todas las operaciones.

Lista de control del proyecto

- [ ] Problema, entidad y reglas escritos (Paso 1).
- [ ] Tabla diseñada y SQL probado en DB Browser (Paso 2).

- [ ] API diseñada en una tabla (Paso 3).
- [ ] Servidor mínimo responde `/api/ping` (Paso 5).
- [ ] Las 5 operaciones CRUD funcionan desde la consola del navegador (Paso 7).
- [ ] Los datos inválidos responden `400` (Paso 8).
- [ ] La página lista, crea, edita, elimina y busca (Paso 9).
- [ ] Comentarios y README completos (Paso 10).
- [ ] Compila desde cero en una carpeta limpia (Paso 10).

Fase 7 – Proyecto final

Construya, desde una carpeta vacía, una aplicación web con backend en C++ y base de datos SQLite sobre un tema propio, siguiendo los 10 pasos de la Fase 6.

Requisitos mínimos

- Una tabla con al menos **5 columnas** (sin contar `id`), con al menos 2 tipos de dato distintos y al menos una restricción `CHECK`.
- API REST con las 5 operaciones CRUD y búsqueda.
- Validación en el servidor con respuestas `400` y mensajes claros.
- Consultas SQL **siempre con parámetros `?`** (cero concatenación de texto del usuario).
- Página web que permita listar, crear, editar, eliminar y buscar.
- Al menos una consulta de agregación (`COUNT`, `AVG`, `SUM`…) mostrada en la página.
- Código comentado y `README.md` completo.

Ideas de temas

Tema	Entidad	Agregación sugerida
Biblioteca escolar	Libro	Libros prestados vs. disponibles
Inventario de una soda o pulpería	Producto	Valor total del inventario (`SUM(precio * cantidad)`)
Control de citas de un consultorio	Cita	Citas por día
Torneo deportivo	Equipo o partido	Tabla de posiciones

Tema	Entidad	Agregación sugerida
Gastos personales	Gasto	Total por categoría (`GROUP BY`)
Adopción de mascotas	Mascota	Mascotas por especie
Colección de películas o videojuegos	Título	Calificación promedio por género

Retos opcionales (puntos extra)

- Segunda tabla relacionada con `FOREIGN KEY` y una consulta con `JOIN`.
- Ordenar la tabla por columna (con lista blanca) o paginación.
- Exportar los datos a un archivo CSV desde una ruta del servidor.

Entregables

1. La carpeta del proyecto comprimida en `.zip`, **sin** las carpetas `build/` ni `data/`.
2. `README.md` con las tablas de diseño (base de datos y API) de los Pasos 2 y 3.
3. Una demostración en vivo de 5 minutos: mostrar cada operación y explicar el recorrido de una petición por las capas.

Rúbrica (100 puntos)

Criterio	Excelente	Suficiente	Insuficiente	Puntos
Diseño (tabla y API)	Tablas completas, tipos y restricciones adecuados, rutas REST correctas	Diseño con errores menores	Sin diseño previo o incoherente	15
Base de datos y SQL	CRUD completo, parámetros `?` en todas las consultas, agregación funcional	Falta una operación o hay una consulta concatenada	Varias operaciones no funcionan	20

Criterio	Excelente	Suficiente	Insuficiente	Puntos
Servidor y API	Todas las rutas con códigos HTTP correctos	Códigos HTTP incorrectos en algunas rutas	Rutas faltantes o que fallan	20
Validación y seguridad	Valida todos los campos en el servidor, usa `textContent`	Validación parcial	Sin validación en el servidor	15
Frontend	Todas las operaciones desde la página, mensajes de error visibles	Falta una operación en la página	La página no se comunica con la API	15
Documentación	README completo y código comentado con claridad	README incompleto o pocos comentarios	Sin documentación	10
Presentación	Explica con seguridad el recorrido de una petición	Explica con ayuda	No logra explicar su código	5

Anexos

A. Glosario

Término	Significado
API	Lista de peticiones que un servidor sabe atender
Backend	La parte del programa que corre en el servidor (aquí, C++)
CRUD	Create, Read, Update, Delete: las cuatro operaciones básicas sobre datos
Enlazador	Programa que junta los archivos `.o` en un `.exe`

Término	Significado
Frontend	La parte que corre en el navegador (HTML, CSS, JavaScript)
Inyección SQL	Ataque que mete código SQL dentro de un dato; se evita con parámetros `?`
JSON	Formato de texto para intercambiar datos (`{ }` objeto, `[ ]` arreglo)
Lambda	Función sin nombre escrita en el lugar donde se usa
localhost	"Esta misma computadora"
Modelo	`struct` que representa una fila de una tabla
Mutex	Candado que impide que dos hilos usen un recurso al mismo tiempo
Parámetro SQL	Espacio `?` en una consulta que se rellena con `bind`
Puerto	Número de "puerta" por la que un servidor escucha (8080)
RAII	Técnica de C++: el constructor toma un recurso y el destructor lo libera
REST	Estilo de API: una ruta por tipo de dato y métodos HTTP para la acción
XSS	Ataque que mete código HTML/JS en la página; se evita con `textContent`

B. Solución de problemas

Problema	Causa probable	Solución
`g++ no se reconoce como un comando`	g++ no está en el PATH	Repita la Fase 0, Paso 2, y abra una terminal nueva
`No se encontro public/index.html`	El servidor se ejecutó desde otra carpeta	Use `iniciar.bat`

Problema	Causa probable	Solución
`No se pudo abrir el puerto 8080`	Ya hay un servidor corriendo	Cierre la otra ventana negra o use `build\servidor.exe 3000`
`ld: cannot open output file ... Permission denied` al compilar	El servidor sigue corriendo y Windows bloquea el `.exe`	Deténgalo con `Ctrl + C` antes de compilar
La página dice "No se pudo conectar con el servidor"	El servidor no está corriendo	Revise la ventana negra
`no such column: ...`	Agregó una columna pero la tabla vieja sigue en el `.db`	Borre `data/escuela.db`
Cambié el C++ y no veo el cambio	No recompiló o no reinició el servidor	`Ctrl + C`, `compilar.bat`, `iniciar.bat`
Cambié el JS/CSS y no veo el cambio	El navegador guardó la versión vieja	`Ctrl + F5`
Muchos errores al compilar	Normalmente uno causa los demás	Corrija solo el **primer** `error:` y vuelva a compilar

C. Referencia rápida

SQL

```
CREATE TABLE IF NOT EXISTS t (id INTEGER PRIMARY KEY AUTOINCREMENT, nombre
TEXT NOT NULL);
INSERT INTO t (nombre) VALUES (?);
SELECT * FROM t WHERE nombre LIKE ? ORDER BY nombre LIMIT ? OFFSET ?;
UPDATE t SET nombre = ? WHERE id = ?;
DELETE FROM t WHERE id = ?;
SELECT COUNT(*), AVG(x), SUM(x), MIN(x), MAX(x) FROM t;
SELECT grupo, COUNT(*) FROM t GROUP BY grupo;
```

SQLite desde C++ (con la clase `Sentencia` del demo)

```
Sentencia s(db_, "SELECT id, nombre FROM t WHERE id = ?");
s.bind(1, id);
while (s.siguiente()) {
    int id = s.entero(0);
    std::string nombre = s.texto(1);
}
```

Rutas con cpp-httplib

```
svr.Get("/api/cosas", [&](const httplib::Request& req, httplib::Response&
res) { ... });
svr.Post("/api/cosas", ...);                  // datos en req.body
svr.Put(R"(/api/cosas/(\d+))", ...);           // id en req.matches[1]
svr.Delete(R"(/api/cosas/(\d+))", ...);
req.get_param_value("q");                     // lee ?q=...
```

JSON con nlohmann/json

```
json j = json::parse(req.body, nullptr, false);   // no lanza excepción
if (j.is_discarded()) { /* JSON inválido */ }
std::string nombre = j["nombre"].get<std::string>();
res.set_content(j.dump(), "application/json");
```

fetch en JavaScript

```
const lista = await (await fetch('/api/cosas')).json();
await fetch('/api/cosas/3', { method: 'DELETE' });
await fetch('/api/cosas', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ nombre: 'algo' })
});
```

D. Documentación oficial

- SQLite: lenguaje SQL
- SQLite: interfaz en C
- cpp-httplib
- nlohmann/json
- MDN: usar fetch

- MDN: códigos de estado HTTP